

# Loss Functions & Metrics, Under/Overfitting, Regularization

Sattam Altwaim

sattam.tuuaim@kaust.edu.sa



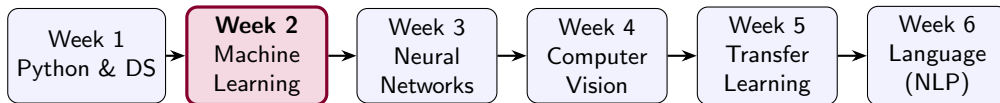
جامعة الملك عبد الله  
للعلوم والتقنية  
King Abdullah University of  
Science and Technology



KAUST Academy & Mawhiba  
AI Olympiad Training (Beginners)  
Week 2, Day 09, Lecture

27 September 2026

1. Big Picture and Motivation
2. Learning Outcomes
3. Loss Functions and Metrics
4. Underfitting and Overfitting
5. Regularization
6. Olympiad Practice
7. Limitations and What Comes Next
8. Summary
9. References



Today: Day 9 of 42

- ▶ **Yesterday:** a model is a function with dials; gradient descent turns the dials to shrink a loss; `fit` and `predict` in `scikit-learn`; a line for numbers, a sigmoid for yes/no.
- ▶ **Today:** how to **judge** a model honestly: which number to report, why training error lies, and how to stop a model from memorising.
- ▶ **Tomorrow:** the other classic models (KNN, SVM, trees, forests). Everything you learn today is how you will compare them.

In yesterday's labs we already kept rows aside and scored on them. Three results came out that we could not explain:

| Where             | What we saw                                        | The open question                                      |
|-------------------|----------------------------------------------------|--------------------------------------------------------|
| Lab 1, used cars  | MAE 20,218 but RMSE 35,787 on the same predictions | which one do we report, and why are they so different? |
| Lab 3, admissions | four grades gave accuracy 0.705                    | is 0.705 good? It equals guessing "no" for everyone    |
| Lab 3, admissions | the organisers score with ROC-AUC                  | what is wrong with accuracy?                           |
| Lab 1, used cars  | 69 columns beat 12 on validation                   | would 500 columns be better still?                     |

## The problem

A model can look excellent by the wrong number and be useless by the right one.

- ▶ Predict “not admitted” for every student: **accuracy 0.702**, and the model has learned nothing.
- ▶ Fit a degree-12 curve to 40 Chevrolet Tahoe prices: training MAE **20,629**, validation MAE **57,081**. The best-looking model on the training rows is the worst on new cars.
- ▶ Report MSE on the used cars and one 1,150,000 SAR Mercedes decides the score for 5,000 ordinary cars.

Yesterday's toolkit measures how well the dials fit the *training* rows. Nothing in it says how the model will do on Friday's hidden test set, which is the only score that counts.

## In one sentence

Separate the **loss** the model minimises from the **metric** the judge reads, measure every metric on rows the model never saw, and when the model fits the training rows better than the new ones, **simplify it or penalise it**.

- ▶ Three ideas, each one section: the right **metric** for the task, **under- and overfitting** with the train/validation gap as the alarm, and **regularization** as the repair.
- ▶ Every number today comes from the two tables you already know: the used cars and the admissions.

## Where you will meet it

Every Olympiad task states its metric in one line (“scored by MAE”, “by macro F1”, “by ROC-AUC”). Reading that line correctly, and validating on held-out rows before submitting, is worth more points than any model choice.

By the end of this session, you will be able to:

- ▶ **Distinguish** the loss a model trains on from the metric a task is scored by, and name both for regression and for classification.
- ▶ **Compute** MAE, RMSE and  $R^2$ , and a confusion matrix with accuracy, precision, recall and F1, from a handful of predictions.
- ▶ **Choose** the metric that fits a task, and explain when accuracy lies and why ROC-AUC needs probabilities.
- ▶ **Diagnose** underfitting and overfitting from the training and validation errors.
- ▶ **Split** data correctly (train, validation, test) and estimate a score with k-fold cross-validation.
- ▶ **Apply** Ridge and Lasso regularization and pick the penalty strength on validation data.

# Metrics: **Loss Functions and Metrics**

## Definition

The **loss** is the number the model minimises while training; it must be smooth so gradient descent can follow it. The **metric** is the number the task is scored by; it only has to be meaningful to a person.

| Task                      | Loss (inside fit)      | Metric (on the leaderboard)              |
|---------------------------|------------------------|------------------------------------------|
| Regression                | MSE                    | MAE, RMSE, $R^2$                         |
| Binary classification     | log loss               | accuracy, precision, recall, F1, ROC-AUC |
| Multiclass classification | cross-entropy (Week 3) | accuracy, macro F1                       |

- ▶ `LinearRegression` always minimises MSE, even when the task is scored by MAE. `LogisticRegression` always minimises log loss, even when the score is accuracy.
- ▶ The judge's metric is written in the task statement. Compute exactly that one on your validation rows.

Three points with truth  $y = (3, 5, 7)$ , mean  $\bar{y} = 5$ , predictions  $\hat{y} = (2.5, 5.5, 6.5)$ , errors  $e = (-0.5, 0.5, -0.5)$ :

$$\text{MAE} = \frac{1}{n} \sum |e_i| = \frac{0.5+0.5+0.5}{3} = 0.5$$

average distance, same unit as  $y$

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum e_i^2} = \sqrt{\frac{0.25+0.25+0.25}{3}} = 0.5$$

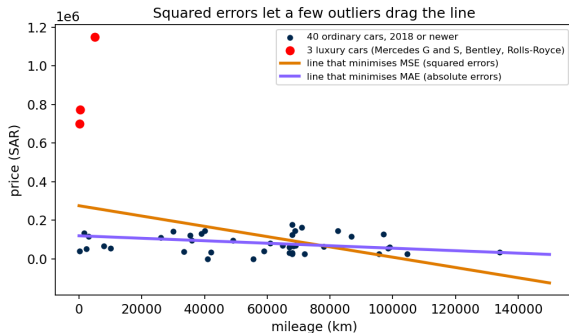
root of MSE, back in the unit of  $y$

$$R^2 = 1 - \frac{\sum e_i^2}{\sum (y_i - \bar{y})^2} = 1 - \frac{0.75}{4 + 0 + 4} = 0.906$$

share of the spread explained; 1 is perfect, 0 is 'none'

- ▶ MAE and RMSE agree here because all errors are equal. With one big error they split: errors  $(0, 0, 3)$  give MAE 1.0 but RMSE 1.73.
- ▶  $R^2$  can go below 0: a model worse than predicting the mean. On the used cars, three numeric columns gave 0.22, the log-price model 0.75.

# MAE or RMSE? Squares Fear Outliers



Source: 40 ordinary and 3 luxury listings from Kaggle  
turkibintalib/saudi-arabia-used-cars-dataset; MAE line by QuantileRegressor(0.5);  
scripts/day09\_metrics\_figures.py.

- ▶ Three luxury cars among 43. The MSE line bends towards them: it predicts 275,697 SAR for a car with 0 km, and negative prices past 110,000 km.
- ▶ The MAE line ignores them: on the 40 ordinary cars its error is 39,714 against 83,669 for the MSE line.

## Rule

Report MAE when a few extreme rows should not dominate; RMSE when big mistakes must be punished extra. If the task says which, use that one.

For one sample with truth  $y \in \{0, 1\}$  and predicted probability  $p$  of “yes”:

$$\ell = -[y \log p + (1 - y) \log(1 - p)]$$

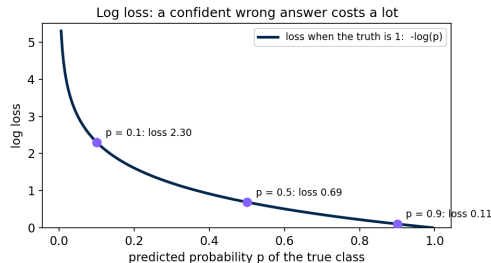
Only one of the two terms survives:  $-\log p$  when  $y = 1$ ,  $-\log(1 - p)$  when  $y = 0$ .

## Worked example

Admitted ( $y = 1$ ), model says  $p = 0.9$ :  
 $\ell = -\log 0.9 = 0.105$ .

Not admitted ( $y = 0$ ), model says  $p = 0.9$ :  $\ell = -\log 0.1 = 2.303$ .

Same confidence, 22 times the loss when it is wrong.



Source:  $-\log p$  on 300 points; `scripts/day09_metrics_figures.py`.

The average  $\ell$  over the training rows is what `LogisticRegression` minimises. It is smooth, so gradient descent can follow it; accuracy is a staircase, so it cannot.

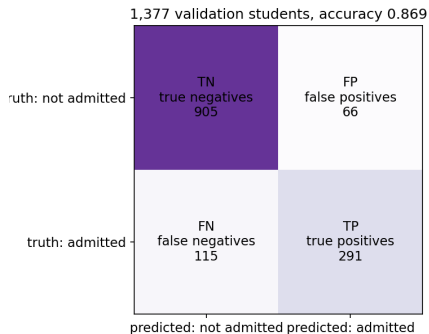
## Definition

**Accuracy** is the share of rows the model labels correctly. It is the right metric only when the classes are about equally common and every mistake costs the same.

- ▶ Admissions: 70.2% of students are not admitted. “Nobody gets in” scores **0.702** without looking at a single column. The four-grade model of Lab 3 scored 0.705: it had learned nothing.
- ▶ A disease that 1 patient in 100 has: “everyone is healthy” scores 0.99 and misses every sick patient.
- ▶ The first number to compute on any classification task is the **majority baseline**, the accuracy of always answering the most common class. A model below it is broken.

## The common mistake

Reporting accuracy on an imbalanced target and calling 0.95 “excellent”. Ask first: what would 0.95 be if I predicted the majority class?



Source: HAIQ 2025 admissions data, logistic regression of Day 08's Lab 3, 20% validation split; `scripts/day09.metrics_figures.py`.

Four counts say everything a single accuracy hides. “Positive” is the class we care about: admitted.

- ▶ **TP** 291: admitted, and the model said admitted.
- ▶ **TN** 905: not admitted, and the model said so.
- ▶ **FP** 66: the model said admitted, but they were not (a false alarm).
- ▶ **FN** 115: admitted, but the model said no (a miss).

$$\text{accuracy} = \frac{\text{TP} + \text{TN}}{\text{all}} = \frac{291 + 905}{1377} = 0.869$$

`confusion_matrix(y_val, pred)` returns these four numbers in this layout.

$$\text{precision} = \frac{TP}{TP + FP} = \frac{291}{291 + 66} = 0.815$$

of those the model admitted, how many

$$\text{recall} = \frac{TP}{TP + FN} = \frac{291}{291 + 115} = 0.717$$

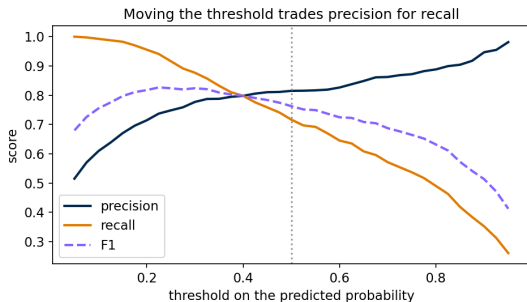
of those really admitted, how many the r

$$F1 = \frac{2 \cdot \text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}} = \frac{2 \cdot 0.815 \cdot 0.717}{0.815 + 0.717} = 0.763$$

one number that is high only if both are

- ▶ “Nobody gets in” has accuracy 0.702 but recall 0 and F1 0: the metric now exposes it.
- ▶ Which one matters depends on the cost of each mistake: a spam filter wants precision (never hide a real email), a disease test wants recall (never miss a sick patient).
- ▶ With three or more classes, **macro F1** is the F1 of each class averaged, so a rare class counts as much as a common one. Many Olympiad tasks score with it.

# The Threshold, and Why ROC-AUC Wants Probabilities



Source: the same admissions model, thresholds 0.05 to 0.95;  
scripts/day09\_metrics\_figures.py.

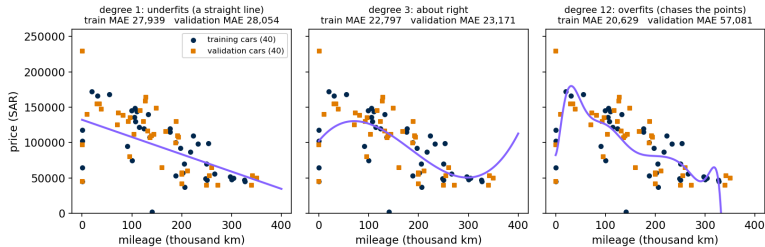
- ▶ predict says “admitted” when  $p \geq 0.5$ . Cut at 0.3: recall 0.877, precision 0.777. Cut at 0.7: precision 0.862, recall 0.571. Every F1 is a score *at one threshold*.
- ▶ **ROC-AUC** scores the probabilities: the chance that a random admitted student gets a higher  $p$  than a random rejected one. 0.5 is coin-flipping, 1.0 perfect; here 0.954.

## The trap

ROC-AUC needs `predict_proba`, not `predict`: 0/1 labels throw the ranking away.

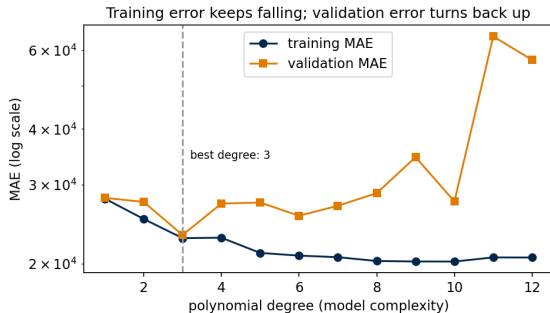
# Metrics: **Underfitting and Overfitting**

# Two Ways to Be Wrong



Source: 80 Chevrolet Tahoe listings, Kaggle turkibintalib/saudi-arabia-used-cars-dataset, half for training and half for validation; polynomial features of the mileage; scripts/day09\_metrics\_figures.py.

- ▶ **Underfitting** (left): too simple for the pattern; both errors high (27,939 and 28,054). A line cannot bend, prices do.
- ▶ **Overfitting** (right): so flexible it follows the noise of the 40 training cars; lowest training error (20,629), highest validation error (57,081).
- ▶ **About right** (middle): smallest validation error, 23,171, and a small gap.



Source: the same 80 Tahoe listings, degrees 1 to 12; scripts/day09\_metrics\_figures.py.

|              | train err. | valid. err.            |
|--------------|------------|------------------------|
| underfitting | high       | high, close to train   |
| about right  | low        | low, close to train    |
| overfitting  | very low   | much higher than train |

- ▶ Training error **only goes down** as the model gets more flexible. It can never tell you to stop.
- ▶ Validation error goes down, then **turns back up**. The turning point is the model to keep.
- ▶ More columns, higher degree, deeper trees, longer training: all move you to the right on this plot.

| Set        | Who sees the labels | Used for                                        | Typical share        |
|------------|---------------------|-------------------------------------------------|----------------------|
| training   | the model, in fit   | turning the dials                               | 60 to 80%            |
| validation | you                 | choosing degree, alpha, features, the threshold | 20%                  |
| test       | the judge only      | the final score, once                           | Friday's hidden file |

```
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2,  
        random_state=42)  
scaler = StandardScaler().fit(X_train)          # learn the scaling on  
        train only  
model.fit(scaler.transform(X_train), y_train)  
score = f1_score(y_val, model.predict(scaler.transform(X_val)))
```

## Leakage

Anything fitted on the validation rows (a scaler, a filled median, a model) makes the validation score a lie. Fit every step on the training rows and only *apply* it to the rest.

|        |       |       |       |       |       |         |
|--------|-------|-------|-------|-------|-------|---------|
| fold 1 | val   | train | train | train | train | score 1 |
| fold 2 | train | val   | train | train | train | score 2 |
| fold 3 | train | train | val   | train | train | score 3 |
| fold 4 | train | train | train | val   | train | score 4 |
| fold 5 | train | train | train | train | val   | score 5 |

5 scores, mean and spread

One split gives one score, and a lucky split moves it. **k-fold cross-validation** cuts the data into  $k$  pieces and lets each be the validation set once.

```
from sklearn.model_selection import
    cross_val_score
pipe = make_pipeline(StandardScaler(),
                      LogisticRegression(
                          max_iter=1000))
scores = cross_val_score(pipe, X, y, cv=5)
print(scores.round(3), scores.mean(),
      scores.std())
# [0.881 0.869 0.870 0.870 0.869] 0.872
0.005
```

The spread (0.005) says which differences between models are real. The pipeline refits the scaler inside every fold: no leakage. Day 13 goes deeper.

# Metrics: **Regularization**

## Definition

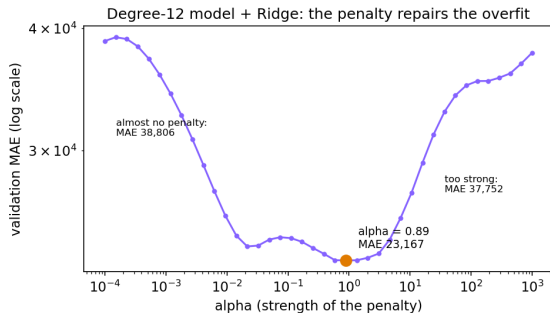
**Regularization** adds a penalty on the size of the weights to the loss: a large weight is allowed only when it buys a real drop in training error.

$$L_{\text{ridge}}(\mathbf{w}, b) = \underbrace{\frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2}_{\text{MSE, as yesterday}} + \underbrace{\alpha \sum_{j=1}^d w_j^2}_{\text{penalty}}$$

$\alpha \geq 0$  is the penalty strength, a dial *you* set:  $\alpha = 0$  is plain linear regression, a huge  $\alpha$  pushes every weight to 0. The bias  $b$  is never penalised.

## Worked example

$\mathbf{w} = (2, 1)$ , MSE 0.5,  $\alpha = 0.1$ : penalty  $0.1 \cdot (4 + 1) = 0.5$ , total 1.0. Shrunk to  $(1.5, 0.8)$  with MSE 0.6: penalty  $0.1 \cdot (2.25 + 0.64) = 0.289$ , total 0.889. The shrunk pair wins: a little more training error, much smaller weights.



Source: the 80 Tahoe listings, degree-12 features scaled, Ridge for 40 values of alpha;  
scripts/day09\_metrics\_figures.py.

- ▶ The wiggles of the overfitted curve need huge weights that cancel each other. The penalty forbids them.
- ▶ Degree 12 with almost no penalty: validation MAE 38,806. With  $\alpha = 0.89$ : **23,167**, as good as the hand-picked degree 3. With  $\alpha = 1000$ : 37,752, now underfitting.
- ▶  $\alpha$  is chosen exactly like the degree: on validation data or by cross-validation, never on the training error (which always prefers  $\alpha = 0$ ).

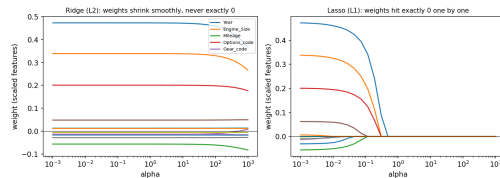
**Ridge (L2):** penalty  $\alpha \sum w_j^2$ . Weights shrink smoothly towards 0 but never reach it. Use it by default.

**Lasso (L1):** penalty  $\alpha \sum |w_j|$ . Weights hit *exactly* 0 one by one as  $\alpha$  grows, so Lasso also **selects features**. On the 12 used-car columns with  $\alpha = 0.05$ , 7 weights are exactly 0; the survivors are Year (0.42), Engine\_Size (0.28), Options (0.17), Mileage and Gulf origin.

## Scale first

The penalty compares weights across columns, so the columns must be on the same scale.

Ridge and Lasso always come after StandardScaler.



Source: Day 08's 12 used-car features (scaled), target log price, 30 values of alpha; scripts/day09\_metrics\_figures.py.

```
from sklearn.linear_model import Ridge, Lasso
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

for alpha in [0.01, 0.1, 1, 10, 100]:
    pipe = make_pipeline(StandardScaler(), Ridge(alpha=alpha))
    pipe.fit(X_train, y_train)
    print(alpha, mean_absolute_error(y_val, pipe.predict(X_val)))

lasso = make_pipeline(StandardScaler(), Lasso(alpha=0.05)).fit(X_train
    , y_train)
print(lasso[-1].coef_)           # zeros mark the dropped
    columns
```

- ▶ Same fit / predict as yesterday; the pipeline scales inside fit so validation rows never leak into the scaler.
- ▶ LogisticRegression is regularized by default: its dial is  $C = 1/\alpha$ , so a *smaller*  $C$  means a *stronger* penalty. Day 11 tunes it.

# Metrics: Olympiad Practice

## Task

The admissions table of Day 08's Lab 3: 6,885 students with 30 columns and a yes/no target, 765 test students. The statement says: *"Metric: ROC-AUC. A score above the baseline is required to qualify."* The baseline is a logistic regression on the single column `school_prestige`. A team submits the 0/1 output of `predict` from a model with validation accuracy 0.87, and scores far below what they expected. Explain what went wrong, and write the three lines that fix it.

- ▶ **Question 1:** the prestige-only baseline has validation accuracy 0.702. Is that model any good? What does its ROC-AUC of about 0.56 add to the picture?
- ▶ **Question 2:** why does a file of 0s and 1s cap the ROC-AUC, and what should the file contain instead?
- ▶ **Question 3:** the team chose its features by watching the training accuracy. Which number should they have watched, and how many times should they have measured it?

Adapted from the Hungarian AI Olympiad 2025 online qualifier "University Admissions" (Hungarian-AI-Olympiad GitHub repository, folder 2025/nyari-online).

**Recognise:** an imbalanced target (29.8% yes), a ranking metric (ROC-AUC), a baseline that equals the majority guess in accuracy.

1. Accuracy 0.702 is the majority baseline, so the prestige model learned almost nothing; its ROC-AUC of 0.56, barely above 0.5, says the same thing in the judge's units.
2. ROC-AUC ranks students by their probability. Labels give only two ranks (all 1s tie, all 0s tie); the ordering inside each group is lost. Submit `predict_proba(...)[ :, 1]`.
3. Watch the validation ROC-AUC, and measure it with 5-fold cross-validation so one lucky split cannot mislead.

```
pipe = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
print(cross_val_score(pipe, X, y, cv=5, scoring="roc_auc").mean())    # about
0.95
pipe.fit(X, y)
pd.DataFrame({"ID": test["ID"], "admitted": pipe.predict_proba(X_test)[ :, 1]})
    .to_csv("submission.csv", index=False)
```

**Check:** the submitted column holds numbers between 0 and 1, not just 0 and 1; the cross-validated ROC-AUC is far above the 0.56 baseline.

# Metrics: **Limitations & What Comes Next**

- ▶ **A validation score is an estimate.** With 40 validation cars, the MAE moves by thousands of SAR between splits. Cross-validation narrows the spread but cannot remove it.
- ▶ **Validation gets used up.** Try 50 models and keep the best validation score: you have started fitting the validation set too. That is why the judge keeps a test set you never see.
- ▶ **Regularization shrinks, it does not fix.** A penalty cannot add the column the model is missing, and it cannot straighten a line for a curved pattern.
- ▶ **The metric is a proxy.** F1 at threshold 0.5 and ROC-AUC can rank two models differently. When the task names one, optimise that one.

## The common mistake

Choosing the threshold, the alpha or the features on the training error. Training error always votes for the most flexible model.

- ▶ **Tomorrow (Day 10):** models that are not lines: k-nearest neighbours, support vector machines, decision trees, random forests and boosting, and how to tune their dials with what you learned today.
- ▶ **Day 11, self-study:** linear and logistic regression in depth, including the derivation behind gradient descent and the C dial of logistic regression.
- ▶ **Day 13, self-study:** cross-validation strategies, PCA and feature importance, and the evaluation habits that win points on Friday.

# Metrics: **Summary**

- ▶ **Loss versus metric:** the model minimises MSE or log loss; the judge reads MAE, RMSE,  $R^2$ , accuracy, F1 or ROC-AUC. Compute the judge's number.
- ▶ **Regression metrics:** MAE is robust, RMSE punishes big errors,  $R^2$  compares with predicting the mean.
- ▶ **Classification metrics:** the majority baseline first; then the confusion matrix, precision, recall, F1; ROC-AUC scores probabilities, so submit `predict_proba`.
- ▶ **Under- and overfitting:** training error only falls; validation error turns back up; the gap is the alarm.
- ▶ **Splits and cross-validation:** fit on train, choose on validation, score once on test; k folds give a mean and a spread.
- ▶ **Regularization:** Ridge shrinks weights, Lasso zeroes them; scale first and choose alpha on validation data.

# Metrics: **References & Resources**

- [1] Bin Talib, T. *Saudi Arabia Used Cars Dataset* (syarah.com listings), Kaggle turkibintalib/saudi-arabia-used-cars-dataset. Used for the outlier, polynomial, Ridge and Lasso figures.  
<https://www.kaggle.com/datasets/turkibintalib/saudi-arabia-used-cars-dataset>
- [2] Hungarian AI Olympiad (2025). *Online qualifier: University Admissions* (task, data, starter and sample solution, CC BY-NC-SA 4.0). Used for the confusion-matrix, threshold and cross-validation figures and the practice task. <https://github.com/Hungarian-AI-Olympiad/HAIO-Hungarian-AI-Olympiad/tree/main/2025/nyari-online>
- [3] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *JMLR* 12. metrics, model\_selection, Ridge, Lasso, QuantileRegressor. <https://scikit-learn.org>
- [4] Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow*, 3rd ed., chapters 2 to 4: metrics, the confusion matrix, learning curves and regularized models.
- [5] All plots: slides-builder/scripts/day09\_metrics\_figures.py (matplotlib, real data only).